

Data Analysis using Python

01 – Introduction

Contents

Shaping, Modelling and Visualizing Data

Introduction to pandas

Introduction to numpy

Introduction to matplotlib

Introduction

Shaping, Modelling and Visualizing Data

Data Analysis can often be thought of as 3 separate activities

Shaping data

This would include performing operations such as
sorting, filtering, ranking

Also would include more sophisticated operations
pivoting, grouping and aggregating

Shaping, Modelling and Visualizing Data

Data Analysis can often be thought of as 3 separate activities

Modelling data

Calculating simple values from a dataset
sum, average, min and max,

Also includes more sophisticated operations
moving averages, interpolation, regression

Shaping, Modelling and Visualizing Data

Data Analysis can often be thought of as 3 separate activities

Visualizing data

Creating simple plots and visualizations

line charts, bar plots, pie charts, histograms and so forth

This would also include more sophisticated charts

3 dimensional plots, choropleth maps, interactive plots

Shaping, Modelling and Visualizing Data

Python is a very mature and resourced language for performing data analysis

Shaping data

The main python used here is **pandas**

Modelling data

The foundational modeling library for python is **numpy**

Visualizing data

The building block for all python visualization libraries is **matplotlib**

pandas

pandas

What is Pandas?

Pandas is a powerful, open-source Python library for data manipulation and analysis.

Built on top of NumPy, it provides easy-to-use structures for handling structured data.

Core Features:

DataFrame: A 2D labelled data structure, akin to a table in a relational database or an Excel spreadsheet.

Series: A 1D labelled array for handling single-dimensional data.

Why Use Pandas?

Simplifies data cleaning, filtering, and transformation.

Supports reading/writing multiple file formats (CSV, Excel, JSON, SQL, etc.).

Provides tools for reshaping and pivoting datasets.

DataFrames

What is a DataFrame?

A 2D labelled data structure in Pandas.

Think of it as a table with rows and columns, similar to a spreadsheet or SQL table.

Key Components:

Rows: Represent observations or records.

Columns: Represent variables or attributes.

Index: A label for rows, enabling efficient lookups and operations.

The diagram illustrates a DataFrame table with the following structure:

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Annotations in the diagram:

- Columns:** A blue label with arrows pointing to the column headers: Name, Team, Number, Position, and Age.
- Rows:** An orange label with arrows pointing to the row indices: 0, 1, 2, 3, 4, 5, and 6.
- Data:** A purple label with a bracket pointing to the data cells of the table, specifically highlighting the values for the first three rows in the 'Number' and 'Position' columns.

DataFrames

```
import pandas as pd
```

import pandas as pd

the conventional way to import the Pandas library in Python, assigning it the alias `pd` for brevity.
allows users to access all Pandas functionalities, such as data manipulation, analysis, and visualization, using concise syntax (e.g., `pd.DataFrame`).

```
df = pd.read_csv(filepath_or_buffer='../Data/constituents.csv', index_col='Symbol')
```

read_csv

Reads data from a CSV file into a DataFrame.

Common arguments:

sep: Specify a delimiter (default is `,`).

header: Specify the row to use as column names.

index_col: Set the index column.

```
df.head() # First 5 rows
```

```
df.head(3) # First 3 rows
```

head

Returns the first few rows of the DataFrame (default is 5 rows).

Useful for quickly inspecting the structure and content of the data.

```
df.tail() # Last 5 rows
```

```
df.tail(3) # Last 3 rows
```

tail

Returns the last few rows of the DataFrame (default is 5 rows).

Helps examine data at the end of the dataset.

```
df.describe()
```

describe

Generates summary statistics for numeric columns in the DataFrame.

Includes metrics like count, mean, std, min, and percentiles.

DataFrames – Selecting rows and columns

```
df['Name']
```

```
df[['Name', 'Sector', 'Price']]
```

Selecting columns

Use `[]` - *SQUARE BRACKETS*

Single column: Access a single column by its name.

Multiple columns: Select multiple columns using a list of column names.

```
df.loc['FB']
```

```
df.loc[['IBM', 'MMM', 'C']]
```

Selecting rows – By Index *Label*

Use `.loc[]` - *SQUARE BRACKETS*

Single row: Access a single row by its name.

Multiple rows: Select multiple rows using a list of rows names.

```
df.iloc[10] # row at position 10
```

```
df.iloc[[10, 20, 30]] rows at positions 10, 20 and 30
```

Selecting rows – By Index *Position*

Use `.iloc[]` - *SQUARE BRACKETS*

Single row: Access a single row by its name.

Multiple rows: Select multiple rows using a list of rows names.

numpy

numpy

What is NumPy?

A powerful Python library for numerical computing.

Provides support for large, multi-dimensional arrays and matrices.

Offers mathematical functions to operate on arrays efficiently.

Key Features:

N-dimensional Array (ndarray):

Efficient, homogeneous container for large datasets.

Broadcasting:

Perform element-wise operations without needing matching dimensions.

Linear Algebra:

Built-in support for matrix operations like dot product and inverse.

Statistical Functions:

Compute mean, median, standard deviation, etc.

numpy - arrays

The most important data structure in numpy

A contiguous collection of values

Efficient Storage and Operations:

NumPy arrays (ndarray) are more memory-efficient and faster than Python lists.

They allow for vectorized operations, avoiding loops in Python.

Multi-Dimensional Support:

NumPy arrays can be 1D, 2D, or higher dimensions for complex datasets.

Homogeneous Data:

All elements in a NumPy array must be of the same data type (e.g., int, float).

Numpy arithmetic

```
import numpy as np
```

```
arr1 = np.array ([1,2,3,4,5])
```

```
arr2 = np.array([10,20,30,40,50])
```

```
arr1 + arr2
```

```
arr1 * arr2
```

```
arr2 - arr1
```

```
arr1 < arr2
```

```
arr1 <= arr2
```

```
(arr1 + 12) >= 15
```

```
(arr2 - arr1) == 27
```

```
np.square(6)
```

```
np.square(arr1)
```

```
np.log10(100)
```

```
np.log(arr2)
```

```
np.exp(arr1)
```

import numpy as np

the standard convention for importing the NumPy library in Python, allowing concise usage of its functions and features.

The alias np is widely used to access NumPy functionalities like creating arrays (np.array) and performing mathematical operations (np.mean, np.dot).

np.array

Create 2 arrays – in this case from python lists, there are other arguments

vectorized arithmetic operations

Numpy arrays come with element by element arithmetic operations
No need to iterate or loop over elements

vectorized relational operations

Same for comparison operators

Scientific functions

numpy comes with a whole battery of mathematical functions
logs and exponential functions
trigonometry
advanced statistics
These work on individual values or on arrays of values

numpy - where

The numpy **where** function is an extremely useful function, we will be using it in some financial analysis that we will be doing later on in this course.

np.where is a function that accepts 3 parameters:

- an expression

- a value to return if the expression is True

- a value to return if the expression is False

```
arr1 = np.array ([1,2,3,4,5,6])  
arr2 = np.array ([2,2,4,4,6,6])  
  
np.where(arr1 % 2 == 0, 'Even', 'Odd')  
np.where(arr1 == arr2, 'Matched', 'Did not Match')
```

numpy – and DataFrames

Every column in a DataFrame is a numpy array

Every row in a DataFrame is a numpy array

Any operation a user can perform on a numpy array can be performed on

- a DataFrame row
- a DataFrame column
- an Entire DataFrame

```
np.where(df['Price'] < 100, 'Cheap', 'Expensive')
```

```
np.where( (df['Sector'] == 'Industrials') & (df['Price'] > 50), 'Good', 'Bad')
```

matplotlib

matplotlib

What is Matplotlib?

a popular Python library for creating static, animated, and interactive visualizations.

allows users to create a wide variety of plots, including line charts, bar charts, scatter plots, and more.

Key Features:

Customizable Visualizations:

Modify every aspect of a plot, such as colours, labels, and axes.

Integration with Other Libraries:

Works seamlessly with Pandas, NumPy, and other Python libraries.

Interactive Plotting:

Combine with Jupyter Notebook for interactive data exploration.

matplotlib

```
import matplotlib.pyplot as plt

# Extract the relevant data
cols = ['52 Week High', '52 Week Low']
rows = ['AAPL', 'AMZN', 'NFLX', 'GOOGL']
data = df.loc[rows, cols]

# Create a Matplotlib figure and axes
fig, ax = plt.subplots(figsize=(10, 6)) # Optional: Adjust figure size

# Plot horizontal bars for each column
data.plot.barh(ax=ax)

# Customize the plot
ax.set_title("52 Week Highs and Lows")
ax.set_xlabel("Value")
ax.set_ylabel("Companies")
ax.legend(title="Metrics", loc="lower right") # Optional: Add legend with a title

# Display the plot
plt.tight_layout() # Adjust layout to avoid clipping
plt.show()
```



